

Condition of Parallelism

- To generalise parallel processing, three major areas needs to progress are:
- computation models for parallel computing.
- Interprocessor communication in parallel architectures.
- System integration (for incorporating parallel systems into general computing environments).

Data and resource Dependences

Data Dependence: The ordering relationship between statements is indicated by the data dependence.

Five types of data dependence are defined below:

(1) Flow dependence: A statement S_2 is flow-dependent on statement S_1 if an execution path exists from S_1 to S_2 and if at least one output (variables assigned) of S_1 feeds in as input (operands to be used) to S_2 . Flow dependence is denoted as $S_1 \rightarrow S_2$.

(2) Anti-dependence: Statement S_2 is antidependent on statement S_1 , if S_2 follows S_1 in program order and if the output of S_2 overlaps the input to S_1 . A direct arrow crossed with a bar as in $S_1 - \overline{\rightarrow} S_2$ indicates anti-dependence from S_1 to S_2 .

*Statement $\backslash(S_{\{1\}})$ reads from a location, and subsequently $\backslash(S_{\{2\}})$ writes to the same location. $\backslash(S_{\{2\}})$ cannot finish before $\backslash(S_{\{1\}})$ reads, or $\backslash(S_{\{1\}})$ will get the wrong value.

Example:

$S_{\{1\}}$: Load R1, A (Read A)

$S_{\{2\}}$: Store R2, A (Write A)

$S_{\{2\}}$ is anti-dependent on $S_{\{1\}}$

(3) Output dependence: Two statements are output-dependent if they produce (write) the same output variable. $S1 \rightarrow S2$ indicates output dependence from S1 to S2.

(4) I/O dependence: Read and write are I/O statements. I/O dependence occurs not because the same variable is involved but because the same file is referenced by both I/O statements.

(5) Unknown dependence: The dependence relation between two statements cannot be determined in the following situations:

- The subscript of a variable is itself subscribed (indirect addressing).
- The subscript does not contain the loop index variable.
- A variable appears more than once with subscripts having different coefficients of the loop variable.
- The subscript is nonlinear in the loop index variable.

*When one or more of these conditions exist, a conservative assumption is to claim unknown dependence among the statements involved.

A **dependence graph** is used to describe the dependence relationships between the statements.

Nodes corresponds to the *program statements (instructions)*, and

Directed edges with different **labels** shows the *ordered relation among the statements*.

A statement S2 is **flow dependent** on statement S1 if an execution path exists from S1 to S2 and if at least one output (variables assigned) of S1 feeds in as input (operands to be used) to S2

It is denoted as:

S1 $\longrightarrow$ S2

Example:

S1: Load R1, A	// R1 $\leftarrow$ Memory [A]
S2: ADD R2, R1	// R2 $\leftarrow$ R2 + R1

Statement S2 is **anti dependent** on the statement S1 if S2 follows S1 in the program order and if the output of S2 overlaps the input to S1

It is denoted as:

$S1 \longrightarrow S2$

Example:

S1: Add R2, R1	// R2 $\leftarrow$ R2 + R1
S2: Move R1, R3	// R1 $\leftarrow$ R3

Two statements are **output dependent** if they produce (write) the same output variable.

It is denoted as:

$$S1 \xrightarrow{\ominus} S2$$

Example:

S1: Load R1, A // R1 $\leftarrow$ Memory [A]

S2: Move R1, R3 // R1 $\leftarrow$ R3

Read and write are I/O statements. **I/O dependence** occurs not because the same variable is involved but because the **same file** is referenced by both I/O statements.

Denoted as:



Example:

S1: Read (4), A(I)

S3: Write (4), A(I)

When dependence relation between two statements cannot be determined is known as **Unknown dependency**.

Unknown dependency may exist due to following situations:

- The subscript of a variable is itself subscripted (indirect addressing). Eg: $a(i[j])$

- The subscript does not contain the loop index variable. Eg: $a[]$

- A variable appears more than once with subscripts having different coefficients of the loop variable (that is, different functions of the loop variable). Eg: $a[i]$ and $a[j]$

- The subscript is nonlinear in the loop index variable.

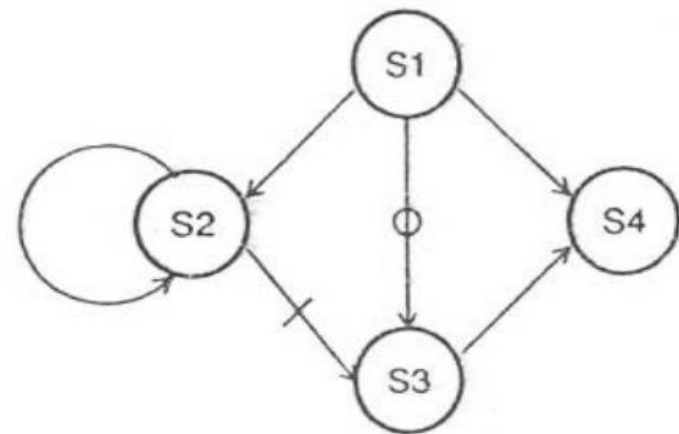
- **Example : Data dependence in programs**

Consider the following code fragment of four instructions:

S1 : Load R1,A	/R1 $\leftarrow$ Memory(A)/
S2 : Add R2,R1	/R2 $\leftarrow$ (R1) +(R2)/
S3 : Move R1 ,R3	/R1 $\leftarrow$ (R3)/
S4 : Store B,R1	/Memory(B) $\leftarrow$ (R1)

As illustrated in diagram (a), S2 is flow-dependent on S1 because the variable A is passed via the register R1. S3 is anti-dependent on S2 because of potential conflicts in register content in R1. S3 is output-dependent on S1 because they both modify the same register R1. Other data dependence relationships can be similarly revealed on a pair wise basis. Note that dependence is a partial ordering relation; that is, the members of not every pair of statements are related.

- For example, the statements S2 and S4 in the above program are totally independent.



(a) Dependence graph

Control dependence

Control dependency arise

- **when the order of execution of statements cannot be determined before runtime.**
 - For example: IF Condition, will not be resolved until runtime.

Control dependence often prohibits parallelism from being exploited.
Compilers are used to eliminate this control dependence and exploit the parallelism.

Example

Control-independent example:

```
for (i=0; i<n; i++) {  
    a[i] = c[i];  
    if (a[i] < 0) a[i] = 1;  
}
```

Control-dependent example:

```
for (i=1; i<n; i++) {  
    if (a[i-1] < 0) a[i] = 1;  
}
```

Resource dependence

Resource dependence is concerned with conflicts in using shared **resources**, such as integer and floating point units (ALUs), registers and memory areas among parallel events, etc.

- ALU conflicts are called **ALU dependence**.
- Memory (storage) conflicts are called **Storage dependence**.

Example:

I1 and I2 both are accessing addition unit, so ALU dependent

Bernstein's Condition

In 1966, Bernstein gave a set of conditions on the basis of which two processes can execute in parallel

Notation:

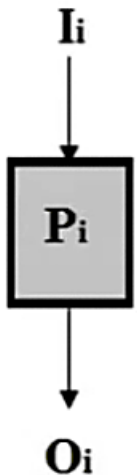
▮ **Process P_i** is a software entity

▮ **Input I_i** is the set of all input variables for a process P_i

▮ I_i is also called the read set or domain of P_i .

▮ **Output O_i** is the set of all output variables for a process P_i .

▮ O_i is also called write set or range of P_i .

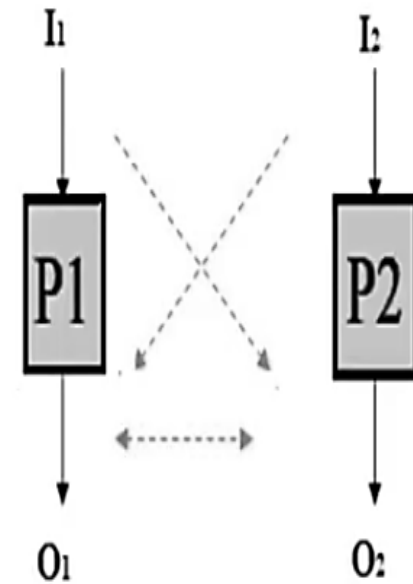


According to Bernstein:

Two processes P_1, P_2 with input sets I_1, I_2 and output sets O_1, O_2 can **execute in parallel** (denoted by $P_1 \parallel P_2$) **if**:

$$\left\{ \begin{array}{l} I_1 \cap O_2 = \emptyset \text{ (null)} \\ I_2 \cap O_1 = \emptyset \\ O_1 \cap O_2 = \emptyset \end{array} \right.$$

These three conditions are known as **Bernstein's Conditions**



In terms of data dependencies,

Bernstein's conditions imply that two processes can execute in parallel if they are:

- flow-**in**dependent,
- anti-**in**dependent, and
- output **in**dependent

$I1 \cap O2 = \emptyset$ **Anti-independent i.e no anti dependence**

$I2 \cap O1 = \emptyset$ **Flow-independent i.e no flow dependence**

$O1 \cap O2 = \emptyset$ **Output-independent i.e. no output dependence**

In general $P_1, P_2, P_3, \dots, P_k$ can execute in parallel if Bernstein's Conditions are satisfied on a pair-wise basis. This means that:

$$P_1 \parallel P_2 \parallel P_3 \parallel \dots \parallel P_k \quad \text{if and only if } P_i \parallel P_j \text{ for all } i \neq j$$

The **properties of parallelism relation** ($\parallel$) is :

- Commutative ($P_i \parallel P_j$ implies $P_j \parallel P_i$),
- Non-transitive ($P_i \parallel P_j$ and $P_j \parallel P_k$, does not imply $P_i \parallel P_k$)
- Associative ($(P_i \parallel P_j) \parallel P_k = P_i \parallel (P_j \parallel P_k)$)

Note: Condition $I_1 \cap I_2 \neq \emptyset$ does not prevent parallelism between two processes. However violation of any of the three conditions prohibits parallelism between two processes

Consider following statements:

$$a = x + y;$$

$$b = x + z;$$

Do they satisfy Bernstein's Conditions?

Solution:

$$I_1 = \{x, y\}$$

$$O_1 = \{a\}$$

$$I_2 = \{x, z\}$$

$$O_2 = \{b\}$$

Here, $I_1 \cap O_2 = \emptyset$

$$I_2 \cap O_1 = \emptyset$$

$$O_1 \cap O_2 = \emptyset$$

So, Bernstein's Conditions are satisfied.

Hence, these two statements can be executed in parallel.

Consider following statements:

$$a = x + y;$$

$$b = a + b;$$

Do they satisfy Bernstein's Conditions?

Solution:

$$\text{Here, } I_1 = \{x, y\} \qquad O_1 = \{a\}$$

$$I_2 = \{a, b\} \qquad O_2 = \{b\}$$

$$\text{Here, } I_1 \cap O_2 = \emptyset$$

$$I_2 \cap O_1 \neq \emptyset$$

$$O_1 \cap O_2 = \emptyset$$

So, Bernstein's Conditions are not satisfied.

Hence, these two statements cannot be executed in parallel.

Consider the following instructions P1, P2, P3, P4, P5 are in program order. Assume each instruction requires one step to execute, no pipelining is considered here.

A) Detect if parallelism is possible using Bernstein's Conditions

B) Draw Dependence graph and Resource Dependence graph

C) Find how many steps are needed in case of sequential execution and parallel execution if two adders are available.

$$P1: C = D \times E$$

$$P2: M = G + C$$

$$P3: A = B + C$$

$$P4: C = L + M$$

$$P5: F = G \div E$$